FutsalPro — Guía de equipo para pruebas (Antigravity + Azure DevOps)

Repo: https://dev.azure.com/daxascencio1/_git/Testeo
Integrantes: Carlos Pérez, Anderson Pineda, Franklin Pérez, Dax Ascencio

Esta guía NO escribe las pruebas por ustedes: da el reparto, la configuración y **prompts para que cada uno trabaje su parte con Antigravity** (y revise lo que genera).

---

1. Reparto por integrante (sacado de los 3 Excel)

Integrante	Unitarias (2xxx)	Aceptación (1xxx)	API / Integración (CPxx)	Carga/Estrés (opcional)
Carlos	2009–2012 (cancelar reserva), 2026–2029 (AuthService)	1001, 1002, 1008, 1009, 1010	CP01–CP04, CP15–CP17	Login (módulo Auth)
Anderson	2001–2008 (crear reserva)	1003–1007, 1020	CP05–CP14	Reservas (GET/POST)
Franklin	2013 (listar activas), 2014–2019 (torneos/ fixture/goles)	1013–1018	CP18–CP23	Torneos (GET público)
Dax	2020–2025 (ganador /penales/cancelar torneo), 2030 (email minúsculas)	1011, 1012, 1019, 1021, 1022	CP24–CP30	Prueba de estrés global + polling de fixture

Responsables de API tomados de `FutsalPro_Pruebas_de_API.xlsx` (la versión con responsables por caso).

Orden de dependencia: Unitarias → Integración (API) → Aceptación → Carga/Estrés.

---

2. Conectar Antigravity con Azure DevOps

Hay dos conexiones distintas. Necesitan las dos.

2.1 Git (clonar y subir código) — lo hace cada integrante

a) Acceso al proyecto (lo hace Dax, dueño de la organización):

1. `https://dev.azure.com/daxascencio1` → Organization settings → Users → **Add users**.
2. Agregar el correo de cada compañero, nivel **Basic**, y agregarlo al proyecto (Testeo).
3. Project settings → Repositories → Testeo → Security: dar **Contribute** (y *Create branch*) al grupo del proyecto.

b) Cada integrante clona con Antigravity/terminal:

```
git clone https://daxascencio1@dev.azure.com/daxascencio1/Testeo/_git/Testeo
cd Testeo
git config user.name  "Tu Nombre"
git config user.email "tu_correo@..."
```

Git Credential Manager abre el navegador para iniciar sesión. Si no, crear un **PAT** (User settings → Personal access tokens, scope *Code: Read & write*) y usarlo como contraseña.

> *Si el nombre del proyecto no es `Testeo`, copien la URL desde el botón **Clone** del repo en Azure.*

2.2 MCP de Azure DevOps (que el agente de Antigravity lea/cree Work Items, repos, Test Plans)

Servidor oficial de Microsoft: `@azure-devops/mcp`. Requiere **Node.js 20+**.

1. En Antigravity abrir el panel del agente → menú **"..." → MCP Servers / Manage MCP Servers → View raw config** (abre `mcp_config.json`).
2. Agregar dentro de `mcpServers`:

```
{
  "mcpServers": {
    "azure-devops": {
      "command": "npx",
      "args": ["-y", "@azure-devops/mcp", "daxascencio1", "--authentication", "interactive"]
    }
  }
}
```

3. Reiniciar Antigravity. La primera vez abre el navegador para iniciar sesión con la cuenta que tiene acceso a la organización.
4. Probar con este prompt: "*Lista los proyectos de la organización daxascencio1 de Azure DevOps.*"

Notas importantes:

- **Dónde guardar el archivo:** el global es `~/.gemini/config/mcp_config.json` (Windows: carpeta de usuario). El de workspace es `.agents/mcp_config.json`. **Si usan PAT, nunca lo pongan en el de workspace dentro del repo** (se subiría a Azure). Agreguen `.agents/` al `.gitignore` por si acaso.
- Si el login interactivo falla (organizaciones con cuenta personal Microsoft a veces lo rechazan), usen un PAT por variable de entorno: `--authentication envvar` y `"env": {"ADO_MCP_AUTH_TOKEN": "<PAT>"}`. Verifiquen la sintaxis exacta en el README de `microsoft/azure-devops-mcp`, puede cambiar entre versiones.
- Alternativa remota sin instalar nada: `https://mcp.dev.azure.com/daxascencio1`, pero exige que la organización esté conectada a Microsoft Entra ID.
- Este MCP puede borrar cosas. Pónganle a cada PAT solo los scopes necesarios: *Code (R/W)*, *Work Items (R/W)*, *Test Management (R/W)*.

---

3. Flujo de ramas

```
main
 └── develop            ← base de todo
      ├── (rama de Carlos)   ya existe
      ├── (rama de Franklin) ya existe     ← ajusten nombres a los reales
      └── develop        ← Dax y el 4.º integrante trabajan aquí
```

Todos primero:

```
git fetch origin
git checkout develop            # si no existe: git checkout -b develop && git push -u origin develop
git pull origin develop
```

Quienes ya tienen rama (traer lo último de develop):

```
git checkout mi-rama
git merge origin/develop        # resolver conflictos si hay
git push
```

Cuando terminen una tarea: **Pull Request mi-rama → develop** en Azure Repos.

Quienes trabajan directo en develop:

```
git pull --rebase origin develop    # SIEMPRE antes de empezar y antes de push
git add .
git commit -m "test(unit): 2020-2022 TorneoService ganador"
git push origin develop
```

Reglas para no pisarse (todos):

1. **Una sola persona crea el andamiaje** (proyectos de prueba, paquetes NuGet, `CustomWebApplicationFactory`) — Prompt 0 — y lo sube **primero**. El resto espera ese commit y hace `pull`.
2. **Cada quien en sus propios archivos.** `ReservaService` lo tocan 3 personas, por eso van **clases de prueba separadas** (ver tabla de §4).
3. Commits pequeños y frecuentes. Mensaje: `test(tipo): ids — descripción`.
4. No tocar `.sln`, `.csproj` ni código de producción sin avisar al grupo.
5. Recomendado: en Azure → Repos → Branches → `develop` → Branch policies: pedir 1 revisor. Opcional.

---

4. Estructura de pruebas (adaptada del ejemplo SupermarketAPI)

El ejemplo usa xUnit + Moq + FluentAssertions + EF Core InMemory + `WebApplicationFactory` (integración) + k6/JMeter (carga). Lo mismo aplica aquí (.NET 8).

```
FutsalPro/
├── <ProyectoApi>/                          (ya existe)
├── FutsalPro.UnitTests/
│   └── Services/
│       ├── Auth/        AuthService_LoginTests.cs (Carlos)   AuthService_EmailTests.cs (Dax)
│       ├── Reservas/    ReservaService_CrearTests.cs (Anderson)
│       │                ReservaService_CancelarTests.cs (Carlos)
│       │                ReservaService_ListarTests.cs (Franklin)
│       └── Torneos/     TorneoService_CrearFixtureTests.cs (Franklin)
│                        TorneoService_ResultadosTests.cs (Dax)
├── FutsalPro.IntegrationTests/
│   ├── Infrastructure/  CustomWebApplicationFactory.cs (Prompt 0)
│   └── Endpoints/       AuthEndpointsTests.cs (Carlos), ReservasEndpointsTests.cs (Anderson+Carlos, 2
                             clases),
│                        TorneosEndpointsTests.cs (Franklin), TorneosResultadosEndpointsTests.cs (Dax)
├── PerformanceTests/    (k6 y/o JMeter, una carpeta por módulo)
├── docs/
│   ├── plan-de-pruebas.md
│   ├── historias-de-usuario.md
│   ├── aceptacion/<integrante>/*.feature + evidencias/
│   └── excel/ (los 3 Excel actualizados)
└── azure-pipelines.yml (opcional)
```

Convención para trazabilidad con los Excel (todos):

```
[Fact(DisplayName = "UT-2009: Cancelación con motivo en blanco lanza BusinessRuleException")]
[Trait("Modulo", "Reservas")]
[Trait("CasoId", "2009")]
[Trait("Responsable", "Carlos")]
```

Así cada uno corre solo lo suyo: `dotnet test --filter "Responsable=Carlos"` o por módulo: `--filter "Modulo=Torneos"`.

---

5. Prompts

Cómo usarlos: abrir el repo en Antigravity, **pegar el Prompt de contexto (P-CTX)** al inicio de cada conversación nueva y luego el prompt de la tarea. Revisen cada archivo generado antes del commit; ustedes responden por el código.

P-CTX — Contexto común (pegar siempre primero)

```
Contexto: proyecto FutsalPro (reservas de canchas de futsal y torneos). Backend .NET 8 Web API con EF Core
                            + SQL Server, controladores REST, servicios ReservaService, TorneoService y
                            AuthService, excepciones BusinessRuleException / NotFoundException /
                            ArgumentException, JWT para auth. Repo en Azure DevOps (rama develop).

Estamos haciendo pruebas de software con estándar ISO/IEC/IEEE 29119: unitarias, integración, aceptación y
                            (opcional) carga/estrés. Somos 4 integrantes y cada uno trabaja SOLO sus
                            casos.

Reglas:
1. ANTES de escribir código, explora el repo y confirma nombres reales de clases, métodos, DTOs, namespaces
                            y mensajes de error. No inventes nombres: si algo no existe, dímelo.
2. No modifiques código de producción ni .sln/.csproj salvo que te lo pida explícitamente.
3. Estilo: xUnit + Moq + FluentAssertions + EF Core InMemory (un DbContext con Guid.NewGuid() como nombre
                            por prueba), patrón Arrange/Act/Assert con comentarios en español.
4. Cada prueba lleva [Fact(DisplayName = "UT-<Id>: ...")] y los Trait Modulo, CasoId y Responsable.
5. Usa fechas relativas (DateTime.Now.AddDays(n)), nunca fechas fijas.
6. Trabaja solo en los archivos que te indique. Antes de terminar ejecuta dotnet build y dotnet test
                            --filter "Responsable=<mi nombre>" y muéstrame el resultado.
7. Primero muéstrame un plan corto (lista de pruebas a crear) y espera mi OK antes de generar los archivos.
```

P-0 — Andamiaje (lo hace UNA sola persona, primero)

```
[pegar P-CTX]
Tarea: crear el andamiaje de pruebas siguiendo la estructura del proyecto de ejemplo SupermarketAPI
                              (carpetas UnitTests e IntegrationTests con la factoría personalizada).

1. Crea FutsalPro.UnitTests y FutsalPro.IntegrationTests (xUnit, net8.0), agrégalos a la solución y
                              referencia el proyecto de la API.
2. Paquetes: UnitTests -> Moq 4.20.x, FluentAssertions 6.12.1, Microsoft.EntityFrameworkCore.InMemory
                              8.0.x. IntegrationTests -> Microsoft.AspNetCore.Mvc.Testing 8.0.x,
                              FluentAssertions 6.12.1, Microsoft.EntityFrameworkCore.InMemory 8.0.x.
3. En IntegrationTests/Infrastructure crea CustomWebApplicationFactory : WebApplicationFactory<Program> que
                              (a) reemplaza el DbContext de SQL Server por InMemory con nombre único por
                              instancia, (b) siembra datos: usuario admin anderson@futsalpro.com con la
                              contraseña y hash que use realmente AuthService, 2 canchas, estados de
                              reserva (Confirmada/Cancelada), estados de torneo y géneros, (c) expone un
                              método GenerateTestJwtToken usando la configuración JWT real del proyecto.
4. Si Program no es accesible, dime la solución mínima (partial class Program) pero NO la apliques sin
                              confirmar.
5. Crea las carpetas vacías de la estructura (con .gitkeep) y agrega un .gitignore que excluya bin/, obj/,
                              TestResults/, .vs/ y .agents/.
6. Agrega UNA prueba de humo en cada proyecto para validar que todo compila.
Verifica con dotnet build y dotnet test. No subas nada a git; yo haré el commit.
```

P-CARLOS — Auth + Cancelación de reservas

```
[pegar P-CTX]
Soy Carlos. Mis casos:
- Unitarias: 2026, 2027, 2028, 2029 (AuthService) y 2009, 2010, 2011, 2012 (cancelación de reserva en
                              ReservaService).
- Integración (API): CP01, CP02, CP03, CP04 (auth/login) y CP15, CP16, CP17 (cancelar reserva).

Paso 1 (unitarias): lee el archivo Excel docs/excel/FutsalPro_Pruebas_Unitarias_Grupo5.xlsx, toma mis filas
                              y crea:
 - FutsalPro.UnitTests/Services/Auth/AuthService_LoginTests.cs (2026-2029; IConfiguration simulado con Moq;
                              para 2028 valida que el token tenga 3 partes con JwtSecurityTokenHandler y el
                              rol Administrador; 2029 verifica la comparación contra hash SHA-256 en Hex y
                              Base64)
 - Services/Reservas/ReservaService_CancelarTests.cs (2009-2012)
Paso 2 (integración): con CustomWebApplicationFactory crea Endpoints/AuthEndpointsTests.cs (CP01-CP04) y
                              Endpoints/ReservasCancelarEndpointsTests.cs (CP15-CP17), reproduciendo status
                              code y mensaje de error exactos de la hoja de API. Los CP15-17 dependen de
                              una reserva creada: créala dentro de la propia prueba (no dependas del orden
                              de ejecución).
Muéstrame primero el plan y espera mi OK.
```

P-ANDERSON — Crear/validar/editar reservas

```
[pegar P-CTX]
Soy Anderson. Mis casos:
- Unitarias: 2001 a 2008 (ReservaService: validaciones de creación).
- Integración (API): CP05 a CP14.

Paso 1: lee docs/excel/FutsalPro_Pruebas_Unitarias_Grupo5.xlsx y crea
                              Services/Reservas/ReservaService_CrearTests.cs con mis 8 casos. Para 2006 y
                              2007 puebla el contexto InMemory con la reserva previa (activa y cancelada).
Paso 2: crea Endpoints/ReservasEndpointsTests.cs (CP05-CP14) con WebApplicationFactory. IMPORTANTE: el
                              Excel de API tiene fechas fijas (2026-10-15 en CP08, 2026-10-16 en CP11) que
                              ya son pasadas o inválidas hoy; usa fechas relativas dentro del rango válido
                              de 10 días.
Agrega además casos frontera que el Excel no tiene: reserva exactamente a 10 días (debe pasar) y a 10 días
                              + 1 minuto (debe fallar). Dime cuáles agregaste para sumarlos al Excel.
Muéstrame primero el plan y espera mi OK.
```

P-FRANKLIN — Torneos y fixture

```
[pegar P-CTX]
Soy Franklin. Mis casos:
- Unitarias: 2013 (GetActivasAsync excluye canceladas) y 2014 a 2019 (TorneoService: rango de fechas,
                              equipos vacíos, fixture 8 equipos, bye con 7, goles inválidos).
- Integración (API): CP18 a CP23.

Paso 1: crea Services/Reservas/ReservaService_ListarTests.cs (2013) y
                              Services/Torneos/TorneoService_CrearFixtureTests.cs (2014-2019).
Paso 2: crea Endpoints/TorneosEndpointsTests.cs (CP18-CP23).
Atención: el rango de anticipación del torneo es de 15 a 30 días según el código, pero los Excel dicen
                              "menos de 14" (2014) y "mayor a 31" (2015). Antes de escribir esas pruebas
                              lee TorneoService y dime cuál es el límite real; escribe pruebas de frontera
                              (día 14, 15, 30 y 31). CP21 usa fechaInicio fija 2026-11-02 que NO está en
                              rango hoy: usa DateTime.Now.AddDays(20).
Muéstrame primero el plan y espera mi OK.
```

P-DAX — Ganador/penales/cancelar torneo + Auth email

```
[pegar P-CTX]
Soy Dax. Mis casos:
- Unitarias: 2020 a 2025 (TorneoService: ganador local/visita, penales, finalizar con pendientes, cancelar
                              sin motivo, equipo inexistente) y 2030 (AuthService: email normalizado a
                              minúsculas).
- Integración (API): CP24 a CP30 (torneo sin equipos, bye con 7 equipos, resultados local/visita, goles
                              negativos, penales, finalizar con pendientes).

Crea Services/Torneos/TorneoService_ResultadosTests.cs (2020-2025) y
                              Services/Auth/AuthService_EmailTests.cs (2030). Crea
                              Endpoints/TorneosResultadosEndpointsTests.cs con CP24-CP30; en CP26-CP30 crea
                              el torneo dentro de la prueba para obtener los partidoId reales, sin depender
                              del orden de ejecución. Verifica que el ganadorId enviado en un empate
                              pertenezca realmente a uno de los dos equipos del partido; si el código no lo
                              valida, no lo arregles: repórtalo para registrarlo como defecto en Azure
                              Boards.
Muéstrame primero el plan y espera mi OK.
```

P-ACEP — Pruebas de aceptación (cada uno con sus IDs 1xxx)

```
[pegar P-CTX]
Tarea: preparar las pruebas de aceptación de mis casos: <poner tus IDs, ver tabla §1>. Usa
                              docs/excel/FutsalPro_Pruebas_Aceptación_Grupo5.xlsx.

Por cada caso genera en docs/aceptacion/<mi-nombre>/:
1. Un archivo .feature en Gherkin en español (Dado / Cuando / Entonces) con la historia de usuario ("Como
                              administrador quiero ... para ..."), criterios de aceptación y el escenario
                              del caso. Una feature por historia, con el ID del Excel en una etiqueta
                              (@1003).
2. Una plantilla de evidencia (evidencias/<ID>.md) con: pasos, resultado esperado, resultado obtenido
                              (vacío), estado, fecha, captura (nombre de archivo).
3. Para los casos de API, una petición lista en un archivo .http (REST Client) para ejecutarlos contra mi
                              entorno local.
Los casos móviles (1011, 1012, 1022) se ejecutan de forma manual en dispositivo/emulador: solo genera el
                              guion y la plantilla de evidencia.
Al final crea docs/aceptacion/<mi-nombre>/RESUMEN.md con tabla ID | Historia | Criterio | Estado. NO
                              marques ningún caso como aprobado: eso lo hago yo al ejecutarlo.
```

P-CARGA — Carga y estrés (opcional, por módulo)

```
[pegar P-CTX]
Tarea (opcional): crear pruebas de carga y estrés de mi módulo (<Auth | Reservas | Torneos | Global>)
                            copiando el enfoque del proyecto de ejemplo SupermarketAPI/PerformanceTests
                            (k6: load_test.js y stress_test.js; también un .jmx si es viable).
- Carpeta: PerformanceTests/<modulo>/
- Carga: rampa 0->50 VUs en 30 s, 100 VUs durante 2 min, bajada 30 s. Umbrales: p95 < 500 ms y fallos < 1 %.
- Estrés: aumentar hasta encontrar el punto de quiebre (por ejemplo 100/300/600/1000 VUs) y documentarlo.
- Flujo: login -> usar el token -> endpoints de mi módulo (solo lectura salvo que uses datos de prueba
                            descartables).
- BASE_URL parametrizable por variable de entorno, por defecto http://localhost:<puerto de launchSettings>.
- Escribe un LEEME.md con cómo ejecutarlo y cómo interpretar p95, tasa de error y throughput.
REGLA: NUNCA apuntar estas pruebas a https://futsal-k08n.onrender.com ni a la BD de MonsterASP (plan
                            gratuito, base compartida, riesgo de dejar caída la app). Solo local contra
                            una BD de pruebas.
```

P-AZURE-1 — Historias de usuario en Azure Boards (una sola persona, vía MCP)

```
Usa el MCP de Azure DevOps en la organización daxascencio1, proyecto "Testeo" (confirma el nombre listando
                            proyectos).
1. Consulta el proceso del proyecto (Agile, Scrum, Basic o CMMI) y los tipos de work item disponibles;
                            adapta los nombres (User Story / Product Backlog Item / Issue).
2. Crea 4 Epics: Autenticación, Reservas, Torneos y Fixture, App Móvil (Espectador).
3. Lee docs/excel/FutsalPro_Pruebas_Aceptación_Grupo5.xlsx. Por cada caso cuyo título empiece con "HU:" o
                            describa una historia, crea un User Story con título, descripción "Como <rol>
                            quiero <acción> para <beneficio>", criterios de aceptación (columna Resultado
                            Esperado), y vincúlalo a su Epic. Asigna el Responsable que indica el Excel
                            (si el integrante no está en el proyecto, déjalo sin asignar y avísame).
4. No dupliques: antes de crear busca si ya existe uno con el mismo título.
5. Muéstrame primero la lista completa en una tabla y espera mi OK antes de crear nada.
6. Al final guarda docs/historias-de-usuario.md con la tabla ID Excel | Work Item | Epic | Responsable.
```

P-AZURE-2 — Test Cases y trazabilidad (cada uno con sus casos)

```
Usa el MCP de Azure DevOps (org daxascencio1, proyecto Testeo).
1. Crea (o reutiliza) un Test Plan "FutsalPro - Ciclo 1" con 3 Test Suites: Unitarias, Integración (API),
                            Aceptación.
2. Para MIS casos (<IDs>) crea Test Cases con pasos y resultado esperado tomados del Excel, y enlázalos al
                            User Story correspondiente (relación "Tests"). Nómbralos con el mismo ID del
                            Excel (ej. "2009 - Cancelación con motivo en blanco").
3. Para las unitarias e integración, escribe en cada Test Case el nombre del test automatizado
                            (DisplayName) para trazabilidad.
4. Muéstrame el plan y espera mi OK antes de crear.
Si alguna herramienta de Test Plans no está disponible en el MCP, dímelo y dame los pasos manuales en la
                            interfaz web.
```

P-AZURE-3 — Documentación y pipeline (opcional, una persona)

```
1. Con base en el plan de pruebas de ejemplo (SupermarketAPI/documentacion/1 - plan_de_pruebas...), crea
                            docs/plan-de-pruebas.md para FutsalPro: alcance, fuera de alcance, objetivos
                            de calidad, roles (los 4 integrantes con su módulo), niveles de prueba,
                            clasificación de defectos, criterios de suspensión y salida, entregables,
                            herramientas y entorno.
2. Crea azure-pipelines.yml: trigger en develop, dotnet restore/build, dotnet test de UnitTests e
                            IntegrationTests con --logger trx y cobertura (coverlet), y
                            PublishTestResults@2 + PublishCodeCoverageResults. Las pruebas NO deben
                            depender de internet ni de SQL Server real.
3. No incluyas secretos en ningún archivo.
```

(Los pipelines en organizaciones nuevas pueden requerir solicitar el *parallelism grant* gratuito de Microsoft; si no, correr las pruebas local y adjuntar el reporte.)

---

6. Problemas encontrados en los Excel (corregir antes de entregar)

1. **Fechas fijas ya vencidas.** CP08 usa 2026-10-15 y CP11 2026-10-16: hoy es 21/10/2026, así que CP08 fallaría como "reserva retroactiva". CP21 usa fechaInicio 2026-11-02 (12 días), fuera del rango de 15–30. Usen fechas relativas.
2. **Rango de torneos inconsistente.** 1014/2014 dicen "menos de 14 días", 2015 dice "mayor a 31", pero el mensaje del sistema dice 15–30. Definan el límite real y agreguen pruebas de frontera.
3. **Casos duplicados.** 1018 (Franklin) y 1019 (Dax) son el mismo escenario de penales; y 2022 / CP29 también. Decidan uno solo o diferencien el alcance.
4. **Usen un solo Excel de API**: el que tiene responsables por caso (`FutsalPro_Pruebas_de_API.xlsx`). Descarten la otra versión.
5. **Hoja de Unitarias** aún tiene `[Nombre del Proyecto]` y `[Descripción del Ciclo de Pruebas]` sin llenar. Fechas de "Última Fecha de Estado" raras (12/10/2027 ... 12/10/2032 en 1013–1018).
6. **Las pruebas API apuntan a Render** (`futsal-k08n.onrender.com`): para integración automatizada usen `WebApplicationFactory` + InMemory (rápido, repetible, sin tocar datos reales). Render solo para una prueba de humo manual.
7. 1012 y 1016 hablan de comportamientos (deep link, "pase directo simulado 3-0") que conviene aclarar con quien implementó el código antes de probarlos.